KIT-KALAIGNARKARUNANIDHI INSTITUTE OF TECHNOLOGY (AN AUTONOMOUS INSTITUTION)
(Accredited by NAAC&NBA with 'A' Grade) Approved by AICTE & Affiliated to Anna University, Chennai)
Kannampalayam Post, Coimbatore-641402

Department of CSE(Artificial Intelligence and Machine Learning)

B25CSI201 – DATA STRUCTURES

NAME : ……

BATCH : …… **REG.NO : ……...**

DEPT : …….. **YEAR : ……**

KIT-KALAIGNARKARUNANIDHI INSTITUTE OF TECHNOLOGY
(ANAUTONOMOUS INSTITUTION)
(AccreditedbyNAAC&NBAwith'A'Grade)
Approved by AICTE & Affiliated to Anna University, Chennai)
Kannampalayam Post, Coimbatore-641402

Department of CSE(Artificial Intelligence and Machine Learning)

BONAFIDE CERTIFICATE

Name :……

RollNo. :…… Reg. No.: ……

Branch :B.E. CSE(Artificial Intelligence and Machine Learning)

Certified that this is Bonafide record work done by Mr./Ms ..... of

…… during the academic year 2025-2026.

FACULTY IN-CHARGE **HOD**

Submitted for the University Practical Examination held on…….

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Instructions for Laboratory Classes

1. Enter the lab with record work book & necessary things.
2. Enter the lab without bags and footwear.
3. Footwear should be kept in the outside shoe rack neatly.
4. Maintain silence during the Lab hours.
5. Read and follow the work instructions inside the laboratory.
6. Handle the computer systems with care.
7. Shutdown the Computer properly and arrange chairs in order before leaving the lab.
8. The program should be written on the left side pages of the record workbook.
9. The record work book should be completed in all aspects and submitted in the next class itself.
10. Experiment number with date should be written at the top left-hand corner of the record work book page.
11. Strictly follow the uniform dress code for Laboratory classes.
12. Maintain punctuality for lab classes.
13. Avoid eatables inside and maintain the cleanliness of the lab.

VISION

To produce competent professionals in Artificial Intelligence and Machine Learning through excellence in education and move the society in interesting and innovative directions.

MISSION

- To Provide technical knowledge on par with Industry to the students through qualified faculty members having knowledge in recent trends and technologies.
- To transform the students into technically proficient professionals in the field of Artificial Intelligence and Machine learning.
- To Provide opportunities for students to improve leadership skills in order to address a variety of societal concerns through team work and innovative projects.

PROGRAMME OUTCOMES (POs)

Students graduating from Computer Science & Engineering (Artificial Intelligence and Machine Learning) should be able to:

PO1 Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex Artificial Intelligence and Machine Learning problems.

PO2 Problem Analysis: Identify, formulate, research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and Artificial Intelligence and Machine Learning.

PO3 Design/Development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations in the field of Artificial Intelligence and Machine Learning.

PO4 Conduct Investigations of Complex Problems: Using research-based knowledge and Artificial Intelligence & Machine Learning oriented research methodologies including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5 Modern Tool Usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex Artificial Intelligence and machine learning engineering activities with an understanding of the limitations.

PO6 The Engineer and Society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7 Environment and Sustainability: Understand the impact of the professional Artificial Intelligence and Machine Learning Engineering solutions in societal and environmental contexts, and demonstrate the knowledge, and need for the sustainable development.

PO8 Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9 Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse teams and in multidisciplinary settings.

PO10 Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11 Project Management and Finance: Demonstrate knowledge and understanding of the Artificial Intelligence and Machine Learning engineering and management principles and apply these to one's own work, as a member and leader in a team and, to manage projects in multidisciplinary environments.

Program Educational Objectives (PEOs)

PEO1: Graduates will be able to apply their technical knowledge to solve real world problems, with adequate leadership qualities.

PEO2: Graduates will have the capability to apply their knowledge and skills to identify the cutting edging research and develop the solutions on problems of social relevance.

PEO3: Graduates will be able to engage in life-long learning by completing advanced software Technologies, certificates, and/or other professional development.

Program Specific Outcomes (PSOs)

PSO1: Apply fundamental concepts of Artificial Intelligence and Machine Learning according to excellence and excitement towards research, to provide elegant solutions to complex problems.

PSO 2: Develop skills to address and solve Artificial Intelligence based social and automate business solutions using cutting edge technologies.

COURSE OUTCOMES:

Students will be able to:

CO	Course Outcome	Knowledge Level
CO1	To introduce fundamental data structures, Including arrays, linked lists, stacks, and queues	K3
CO2	To develop an understanding of the efficiency of algorithms using time and space complexity.	K4
CO3	To study advanced data structures like trees, graphs, and hash tables, and their applications.	K3
CO4	To learn and Implement algorithms for sorting, searching, and manipulating data.	K4
CO5	To encourage problem-solving using data structures In programming tasks.	K3

CO/PO & PSO		PO1 (K3)	PO2 (K4)	PO3 (K3)	PO4 (K5)	PO5 (K6)	PO6 (K3, (A3)	PO7 (K2 (A3)	PO8 (K3, (A3)	PO11 (K3, (A3)	PSO1 (K4,A3)	PSO2 (K3,A3)
CO1	K3	3	3	3	-	-	-	-	-	-	3	3
CO2	K4	3	3	2	2	-	-	1	-	2	3	3
CO3	K3	3	3	3	2	2	-	1	-	2	3	3
CO4	K4	3	3	3	2	2	-	1	-	2	3	3
CO5	K3	3	3	3	2	2	-	1	-	3	3	3
Weighted Average		3	3	3	3	2	2	2	-	2	3	3

INDEX

Sl. No	Name of the Experiment
1	Implement array operations (insertion, deletion, traversal)
2	Implement singly linked list (insert, delete, traverse)
3	Implement doubly linked list (insert, delete, traverse)
4	Implement stack using array and linked list
5	Implement queue using array and linked list
6	Implement binary search tree (insertion, deletion, search)
7	Implement AVL tree (rotation and balancing)
8	Implement graph traversal (DFS, BFS)
9	Implement hash table with collision handling (linear probing, chaining)
10	Implement sorting algorithms (bubble sort, merge sort)

Sl. No.	Date	Name of the Experiment	Page Number	Aim & Algorithm (20 Marks)	Program (30 Marks)	Output & Inference (15Marks)	Viva-Voce (10Marks)	Total (75Marks)	Signature of the Faculty Member

Sl. No.	Date	Name of the Experiment	Page Number	Aim & Algorithm (20 Marks)	Program (30 Marks)	Output & Inference (15Marks)	Viva-Voce (10Marks)	Total (75Marks)	Signature of the Faculty Member

Model Exam Mark (25): _____ Total (100): _____ Signature of the faculty member

Ex.No:	1	Implement array operations (insertion, deletion, traversal)
Date:		

AIM

To implement insertion, deletion, and traversal operations on an array.

ALGORITHM

Step 1: Start the program.

Step 2: Read the size of the array.

Step 3: Read the array elements.

Step 4: Choose the operation (insertion, deletion, traversal).

Step 5: For insertion, read position and element, then shift elements and insert.

Step 6: For deletion, read position and remove the element by shifting elements.

Step 7: Display the array elements.

Step 8: Stop the program.

PROGRAM

```
import java.util.Scanner;

public class ArrayOperations {
  public static void main(String[] args) {
  Scanner sc = new Scanner(System.in);

      int[ ] arr = new int[10];
      int n = 0, choice;

      do {
```

```
System.out.println("\n1.Insert  2.Delete  3.Traverse  4.Exit");
System.out.print("Enter choice: ");
choice = sc.nextInt();

switch (choice) {
    case 1:
        System.out.print("Enter element: ");
        int ele = sc.nextInt();
        System.out.print("Enter position: ");
        int pos = sc.nextInt();

        for (int i = n; i > pos; i--) {
            arr[i] = arr[i - 1];
        }
        arr[pos] = ele;
        n++;
        break;

    case 2:
        System.out.print("Enter position to delete: ");
        pos = sc.nextInt();

        for (int i = pos; i < n - 1; i++) {
            arr[i] = arr[i + 1];
        }
        n--;
        break;

    case 3:
        System.out.print("Array elements: ");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
```

```
                System.out.println();
                break;

            case 4:
                System.out.println("Exit");
                break;
        }
    } while (choice != 4);
  }
}
```

OUTPUT:

INFERENCE:

VIVA QUESTIONS:

1. What is an array?

2. What is array traversal?

3. Why is deletion costly in arrays?

4. What is the index of the first element in an array?

5. What is the time complexity of insertion in an array?

RESULT:

Ex.No:	2	Implement singly linked list (insert, delete, traverse)
Date:		

AIM

To implement insertion, deletion, and traversal operations on a singly linked list.

ALGORITHM

Step 1: Start the program.

Step 2: Create a node with data and next pointer.

Step 3: Insert nodes into the linked list.

Step 4: Delete a specified node from the list.

Step 5: Traverse the linked list.

Step 6: Display the list elements.

Step 7: Stop the program.

PROGRAM

```
import java.util.Scanner;

class SinglyLinkedList {

    static class Node {
        int data;
        Node next;

        Node(int data) {
            this.data = data;
```

```
        this.next = null;
    }
}

Node head;

// Insert at the end
public void insert(int data) {
    Node newNode = new Node(data);
    if (head == null) {
        head = newNode;
        return;
    }
    Node current = head;
    while (current.next != null) {
        current = current.next;
    }
    current.next = newNode;
}

// Delete node by value
public void delete(int key) {
    if (head == null) return;

    if (head.data == key) {
        head = head.next;
        return;
    }

    Node current = head;
    Node prev = null;
    while (current != null && current.data != key) {
        prev = current;
```

```
        current = current.next;
    }

    if (current == null) {
        System.out.println("Element " + key + " not found in the list.");
        return;
    }

    prev.next = current.next;
}

// Traverse and display list
public void traverse() {
    if (head == null) {
        System.out.println("List is empty.");
        return;
    }
    Node current = head;
    System.out.print("Linked List: ");
    while (current != null) {
        System.out.print(current.data + " -> ");
        current = current.next;
    }
    System.out.println("NULL");
}

public static void main(String[] args) {
    SinglyLinkedList list = new SinglyLinkedList();

    // Insert nodes
    list.insert(10);
    list.insert(20);
    list.insert(30);
```

```
        list.traverse();

        // Delete a node
        list.delete(20);
        list.traverse();

        // Delete a non-existing node
        list.delete(50);
    }
}
```

OUTPUT:

INFERENCE:

VIVA QUESTIONS:

1. What is a singly linked list?

2. What are the components of a node?

3. What is the advantage of linked list over array?

4. What is meant by traversal in linked list?

5. What happens when the head node is deleted?

RESULT:

Ex.No:	3	Implement doubly linked list (insert, delete, traverse)
Date:		

AIM

To implement insertion, deletion, and traversal operations on a doubly linked list.

ALGORITHM

Step 1: Start the program.

Step 2: Create a node with previous, data, and next pointers.

Step 3: Insert nodes into the doubly linked list.

Step 4: Delete a specified node from the list.

Step 5: Traverse the list in forward direction.

Step 6: Display the list elements.

Step 7: Stop the program.

PROGRAM

```
import java.util.Scanner;

class DoublyLinkedList {

    // Node class
    class Node {
        int data;
        Node prev;
        Node next;

        Node(int data) {
```

```
        this.data = data;
        this.prev = null;
        this.next = null;
    }
}

Node head = null;

// Insert at end
public void insert(int data) {
    Node newNode = new Node(data);

    if (head == null) {
        head = newNode;
        return;
    }

    Node temp = head;
    while (temp.next != null) {
        temp = temp.next;
    }

    temp.next = newNode;
    newNode.prev = temp;
}

// Delete by value
public void delete(int key) {
    if (head == null) {
        System.out.println("List is empty");
        return;
    }
```

```
    Node temp = head;

    // If head node
    if (temp != null && temp.data == key) {
        head = temp.next;
        if (head != null)
            head.prev = null;
        return;
    }

    while (temp != null && temp.data != key) {
        temp = temp.next;
    }

    if (temp == null) {
        System.out.println("Value not found");
        return;
    }

    if (temp.next != null)
        temp.next.prev = temp.prev;

    if (temp.prev != null)
        temp.prev.next = temp.next;
}

// Traverse
public void traverse() {
    if (head == null) {
        System.out.println("List is empty");
        return;
    }
```

```
        Node temp = head;
        while (temp != null) {
            System.out.print(temp.data + " ");
            temp = temp.next;
        }
        System.out.println();
    }

    // Main method
    public static void main(String[] args) {
        DoublyLinkedList dll = new DoublyLinkedList();

        dll.insert(10);
        dll.insert(20);
        dll.insert(30);

        System.out.println("List after insertion:");
        dll.traverse();

        dll.delete(20);

        System.out.println("List after deleting 20:");
        dll.traverse();
    }
}
```

OUTPUT:

INFERENCE:

VIVA QUESTIONS:

1. What is a doubly linked list?

2. How many pointers does a doubly linked list node have?

3. What is the advantage of doubly linked list over singly linked list?

4. What is backward traversal?

5. What is the extra cost of using a doubly linked list?

RESULT:

Ex.No:	4	Implement stack using array and linked list
Date:		

AIM

To implement stack operations using array and linked list.

ALGORITHM

Step 1: Start the program.

Step 2: Initialize stack using array or linked list.

Step 3: Perform push operation to insert element.

Step 4: Perform pop operation to remove top element.

Step 5: Display stack elements.

Step 6: Stop the program.

PROGRAM

```
import java.util.Scanner;

class StackArray {

    int stack[];
    int top;
    int max;

    StackArray(int size) {
        max = size;
        stack = new int[max];
        top = -1;
    }
```

```
void push(int value) {
    if (top == max - 1) {
        System.out.println("Stack Overflow");
    } else {
        stack[++top] = value;
        System.out.println(value + " pushed");
    }
}

void pop() {
    if (top == -1) {
        System.out.println("Stack Underflow");
    } else {
        System.out.println(stack[top--] + " popped");
    }
}

void display() {
    if (top == -1) {
        System.out.println("Stack Empty");
    } else {
        System.out.println("Stack elements:");
        for (int i = top; i >= 0; i--) {
            System.out.println(stack[i]);
        }
    }
}

public static void main(String args[]) {

    StackArray s = new StackArray(5);
```

```
        s.push(10);
        s.push(20);
        s.push(30);

        s.display();

        s.pop();

        s.display();
    }
}
```

OUTPUT:

INFERENCE:

VIVA QUESTIONS:

1. What is a stack?

2. What are the basic stack operations?

3. What is LIFO principle?

4. What is stack overflow?

5. What is stack underflow?

RESULT:

Ex.No:	5	Implement queue using array and linked list
Date:		

AIM

To implement queue operations using array and linked list.

ALGORITHM

Step 1: Start the program.

Step 2: Initialize queue using array or linked list.

Step 3: Perform enqueue operation to insert element.

Step 4: Perform dequeue operation to remove element.

Step 5: Display queue elements.

Step 6: Stop the program..

PROGRAM

```
import java.util.Scanner;

class QueueImplementation {

    // ----- Queue using Array -----
    int MAX = 5;
    int queue[] = new int[MAX];
    int front = -1, rear = -1;
```

```
void enqueueArray(int value) {
    if (rear == MAX - 1) {
        System.out.println("Array Queue Overflow");
    } else {
        if (front == -1)
            front = 0;
        queue[++rear] = value;
        System.out.println(value + " inserted in Array Queue");
    }
}

void dequeueArray() {
    if (front == -1 || front > rear) {
        System.out.println("Array Queue Underflow");
    } else {
        System.out.println(queue[front++] + " deleted from Array Queue");
    }
}

void displayArray() {
    if (front == -1) {
        System.out.println("Array Queue is empty");
    } else {
        System.out.print("Array Queue: ");
        for (int i = front; i <= rear; i++) {
            System.out.print(queue[i] + " ");
        }
        System.out.println();
    }
}

// ----- Queue using Linked List -----
class Node {
```

```
    int data;
    Node next;

    Node(int value) {
        data = value;
        next = null;
    }
}

Node frontNode = null, rearNode = null;

void enqueueList(int value) {
    Node newNode = new Node(value);

    if (rearNode == null) {
        frontNode = rearNode = newNode;
    } else {
        rearNode.next = newNode;
        rearNode = newNode;
    }

    System.out.println(value + " inserted in Linked List Queue");
}

void dequeueList() {
    if (frontNode == null) {
        System.out.println("Linked List Queue Underflow");
        return;
    }

    System.out.println(frontNode.data + " deleted from Linked List Queue");
    frontNode = frontNode.next;
}
```

```
void displayList() {
    Node temp = frontNode;

    if (temp == null) {
        System.out.println("Linked List Queue is empty");
        return;
    }

    System.out.print("Linked List Queue: ");
    while (temp != null) {
        System.out.print(temp.data + " ");
        temp = temp.next;
    }
    System.out.println();
}

public static void main(String args[]) {

    QueueImplementation q = new QueueImplementation();

    // Array Queue
    q.enqueueArray(10);
    q.enqueueArray(20);
    q.enqueueArray(30);
    q.displayArray();
    q.dequeueArray();
    q.displayArray();

    System.out.println();

    // Linked List Queue
    q.enqueueList(40);
```

```
        q.enqueueList(50);
        q.enqueueList(60);
        q.displayList();
        q.dequeueList();
        q.displayList();
    }
}
```

OUTPUT:

INFERENCE:

VIVA QUESTIONS:

1. What is a queue?

2. What principle does queue follow?

3. What are enqueue and dequeue operations?

4. What is queue overflow?

5. What is the difference between stack and queue?

RESULT:

Ex.No:	6	Implement binary search tree (insertion, deletion, search)
Date:		

AIM

To implement insertion, deletion, and search operations in a Binary Search Tree.

ALGORITHM

Step 1: Start the program.

Step 2: Create a Binary Search Tree node structure.

Step 3: Insert elements into the tree following BST rules.

Step 4: Search for a given element in the tree.

Step 5: Delete a specified node from the tree.

Step 6: Display the tree using inorder traversal.

Step 7: Stop the program.

PROGRAM

```
import java.util.Scanner;
class Node {
    int data;
    Node left, right;

    Node(int value) {
        data = value;
        left = right = null;
    }
```

```
}

class BST {
    Node root;

    // Insert
    Node insert(Node root, int key) {
        if (root == null) {
            root = new Node(key);
            return root;
        }

        if (key < root.data)
            root.left = insert(root.left, key);
        else if (key > root.data)
            root.right = insert(root.right, key);

        return root;
    }

    // Search
    boolean search(Node root, int key) {
        if (root == null)
            return false;

        if (root.data == key)
            return true;

        if (key < root.data)
            return search(root.left, key);
        else
            return search(root.right, key);
    }
```

```
// Find minimum value
Node minValueNode(Node root) {
    Node current = root;
    while (current.left != null)
        current = current.left;
    return current;
}

// Delete
Node delete(Node root, int key) {
    if (root == null)
        return root;

    if (key < root.data)
        root.left = delete(root.left, key);
    else if (key > root.data)
        root.right = delete(root.right, key);
    else {
        // Node with one or no child
        if (root.left == null)
            return root.right;
        else if (root.right == null)
            return root.left;

        // Node with two children
        Node temp = minValueNode(root.right);
        root.data = temp.data;
        root.right = delete(root.right, temp.data);
    }
    return root;
}
```

```
// Inorder Traversal
void inorder(Node root) {
    if (root != null) {
        inorder(root.left);
        System.out.print(root.data + " ");
        inorder(root.right);
    }
}

public static void main(String[] args) {
    BST tree = new BST();

    tree.root = tree.insert(tree.root, 50);
    tree.insert(tree.root, 30);
    tree.insert(tree.root, 70);
    tree.insert(tree.root, 20);
    tree.insert(tree.root, 40);
    tree.insert(tree.root, 60);
    tree.insert(tree.root, 80);

    System.out.println("Inorder traversal:");
    tree.inorder(tree.root);

    System.out.println("\nSearch 40: " + tree.search(tree.root, 40));

    tree.root = tree.delete(tree.root, 20);
    System.out.println("After deleting 20:");
    tree.inorder(tree.root);
}
```

OUTPUT:

INFERENCE:

VIVA QUESTIONS:

1. What is an AVL tree?

2. What is balance factor?

3. Why are rotations required in AVL tree?

4. Name the four imbalance cases in AVL tree.

5. What is the advantage of AVL tree over BST?

RESULT :

.

Ex.No:	7	Implement AVL tree (rotation and balancing)
Date:		

AIM

To implement an AVL tree with rotation and balancing.

ALGORITHM

Step 1: Start the program.

Step 2: Insert elements into the AVL tree.

Step 3: Calculate height and balance factor of nodes.

Step 4: Identify imbalance cases (LL, RR, LR, RL).

Step 5: Perform appropriate rotations.

Step 6: Display the balanced AVL tree.

Step 7: Stop the program.

PROGRAM

```
import java.util.Scanner;

class AVLTree {

    class Node {
        int key, height;
        Node left, right;

        Node(int d) {
            key = d;
            height = 1;
```

```
    }
}

Node root;

int height(Node N) {
    if (N == null)
        return 0;
    return N.height;
}

int max(int a, int b) {
    return (a > b) ? a : b;
}

// Right Rotation (LL Case)
Node rightRotate(Node y) {
    Node x = y.left;
    Node T2 = x.right;

    // Perform rotation
    x.right = y;
    y.left = T2;

    // Update heights
    y.height = max(height(y.left), height(y.right)) + 1;
    x.height = max(height(x.left), height(x.right)) + 1;

    // Return new root
    return x;
}

// Left Rotation (RR Case)
```

```
Node leftRotate(Node x) {
    Node y = x.right;
    Node T2 = y.left;

    // Perform rotation
    y.left = x;
    x.right = T2;

    // Update heights
    x.height = max(height(x.left), height(x.right)) + 1;
    y.height = max(height(y.left), height(y.right)) + 1;

    // Return new root
    return y;
}

// Get Balance Factor
int getBalance(Node N) {
    if (N == null)
        return 0;
    return height(N.left) - height(N.right);
}

// Insert Node
Node insert(Node node, int key) {

    // Step 1: Perform normal BST insertion
    if (node == null)
        return (new Node(key));

    if (key < node.key)
        node.left = insert(node.left, key);
    else if (key > node.key)
```

```
    node.right = insert(node.right, key);
else
    return node; // Duplicate keys not allowed

// Step 2: Update height
node.height = 1 + max(height(node.left), height(node.right));

// Step 3: Get balance factor
int balance = getBalance(node);

// Step 4: Apply rotations

// LL Case
if (balance > 1 && key < node.left.key)
    return rightRotate(node);

// RR Case
if (balance < -1 && key > node.right.key)
    return leftRotate(node);

// LR Case
if (balance > 1 && key > node.left.key) {
    node.left = leftRotate(node.left);
    return rightRotate(node);
}

// RL Case
if (balance < -1 && key < node.right.key) {
    node.right = rightRotate(node.right);
    return leftRotate(node);
}

// Return unchanged node
```

```
        return node;
    }

    // Inorder Traversal
    void inorder(Node node) {
        if (node != null) {
            inorder(node.left);
            System.out.print(node.key + " ");
            inorder(node.right);
        }
    }

    // Main method
    public static void main(String[] args) {
        AVLTree tree = new AVLTree();
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter number of nodes:");
        int n = sc.nextInt();

        System.out.println("Enter the elements:");
        for (int i = 0; i < n; i++) {
            int key = sc.nextInt();
            tree.root = tree.insert(tree.root, key);
        }

        System.out.println("\nInorder Traversal of AVL Tree:");
        tree.inorder(tree.root);

        sc.close();
    }
}
```

OUTPUT:

INFERENCE:

VIVA QUESTIONS:

1. What is an AVL tree?

2. What is balance factor?

3. Why are rotations required in AVL tree?

4. Name the four imbalance cases in AVL tree.

5. What is the advantage of AVL tree over BST?

RESULT:

Ex.No:	8	Implement graph traversal (DFS, BFS)
Date:		

AIM

To implement graph traversal techniques using DFS and BFS.

ALGORITHM

Step 1: Start the program.

Step 2: Read number of vertices and edges.

Step 3: Represent the graph using adjacency matrix or list.

Step 4: Perform Depth First Search from a starting vertex.

Step 5: Perform Breadth First Search using a queue.

Step 6: Display traversal order.

Step 7: Stop the program.

PROGRAM

```
import java.util.*;

public class GraphTraversal {
    private int vertices;
    private LinkedList<Integer> adjList[];

    // Constructor
    GraphTraversal(int v) {
        vertices = v;
        adjList = new LinkedList[v];
```

```
    for (int i = 0; i < v; i++) {
        adjList[i] = new LinkedList<>();
    }
}

// Add edge
void addEdge(int v, int w) {
    adjList[v].add(w);
}

// DFS Traversal
void DFS(int start) {
    boolean visited[] = new boolean[vertices];
    System.out.print("DFS Traversal: ");
    DFSUtil(start, visited);
    System.out.println();
}

void DFSUtil(int v, boolean visited[]) {
    visited[v] = true;
    System.out.print(v + " ");

    Iterator<Integer> i = adjList[v].listIterator();
    while (i.hasNext()) {
        int n = i.next();
        if (!visited[n]) {
            DFSUtil(n, visited);
        }
    }
}

// BFS Traversal
void BFS(int start) {
```

```
    boolean visited[] = new boolean[vertices];
    Queue<Integer> queue = new LinkedList<>();

    visited[start] = true;
    queue.add(start);

    System.out.print("BFS Traversal: ");

    while (!queue.isEmpty()) {
        int s = queue.poll();
        System.out.print(s + " ");

        Iterator<Integer> i = adjList[s].listIterator();
        while (i.hasNext()) {
            int n = i.next();
            if (!visited[n]) {
                visited[n] = true;
                queue.add(n);
            }
        }
    }
    System.out.println();
}

// Main method
public static void main(String args[]) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter number of vertices: ");
    int v = sc.nextInt();

    GraphTraversal g = new GraphTraversal(v);
```

```
        System.out.print("Enter number of edges: ");
        int e = sc.nextInt();

        System.out.println("Enter edges (source destination): ");
        for (int i = 0; i < e; i++) {
            int src = sc.nextInt();
            int dest = sc.nextInt();
            g.addEdge(src, dest);
        }

        System.out.print("Enter starting vertex: ");
        int start = sc.nextInt();

        g.DFS(start);
        g.BFS(start);

        sc.close();
    }
}
```

OUTPUT:

INFERENCE:

VIVA QUESTIONS:

1. What is a graph?

2. What is graph traversal?

3. What is the difference between DFS and BFS?

4. Which data structure is used in BFS?

5. Mention one application of DFS.

RESULT:

Ex.No:	9	Implementation and Comparison of Sorting Algorithms Using Java
Date:		

AIM:

To implement a hash table with collision handling techniques.

ALGORITHM:

Step 1: Start the program.

Step 2: Create a hash table of fixed size.

Step 3: Apply hash function to calculate index.

Step 4: Handle collisions using linear probing or chaining.

Step 5: Insert and search elements in the hash table.

Step 6: Display hash table contents.

Step 7: Stop the program.

PROGRAM

```
import java.util.*;

import java.util.Scanner;

public class HashTable {
    private int size;
    private int[] table;
    private static final int EMPTY = -1;
    private static final int DELETED = -2;

    // Constructor
    public HashTable(int size) {
        this.size = size;
```

```
    table = new int[size];
    for (int i = 0; i < size; i++) {
        table[i] = EMPTY;  // Mark all slots as empty initially
    }
}

// Hash function: Simple modulus function
private int hash(int key) {
    return key % size;
}

// Insert an element into the hash table
public void insert(int key) {
    int index = hash(key);

    // Linear probing for collision handling
    int i = index;
    while (table[i] != EMPTY && table[i] != DELETED) {
        i = (i + 1) % size;  // Move to the next slot
        if (i == index) {
            System.out.println("Hash table is full, cannot insert.");
            return;
        }
    }
    table[i] = key;
    System.out.println("Inserted " + key + " at index " + i);
}

// Search for an element in the hash table
public boolean search(int key) {
    int index = hash(key);
    int i = index;
```

```
    // Linear probing to find the element
    while (table[i] != EMPTY) {
        if (table[i] == key) {
            System.out.println(key + " found at index " + i);
            return true;
        }
        i = (i + 1) % size;
        if (i == index) {
            break; // We looped through the entire table
        }
    }
    System.out.println(key + " not found in the hash table.");
    return false;
}

// Delete an element from the hash table
public void delete(int key) {
    int index = hash(key);
    int i = index;

    while (table[i] != EMPTY) {
        if (table[i] == key) {
            table[i] = DELETED;
            System.out.println(key + " deleted from index " + i);
            return;
        }
        i = (i + 1) % size;
        if (i == index) {
            break;
        }
    }
    System.out.println(key + " not found for deletion.");
}
```

```
// Display the contents of the hash table
public void display() {
    System.out.println("\nHash Table Contents:");
    for (int i = 0; i < size; i++) {
        if (table[i] == EMPTY) {
            System.out.println(i + " -> Empty");
        } else if (table[i] == DELETED) {
            System.out.println(i + " -> Deleted");
        } else {
            System.out.println(i + " -> " + table[i]);
        }
    }
}

// Main method to interact with the user
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter size of hash table: ");
    int size = sc.nextInt();
    HashTable hashTable = new HashTable(size);

    while (true) {
        System.out.println("\nMenu:");
        System.out.println("1. Insert");
        System.out.println("2. Search");
        System.out.println("3. Delete");
        System.out.println("4. Display");
        System.out.println("5. Exit");
        System.out.print("Enter your choice: ");
        int choice = sc.nextInt();
```

```
            switch (choice) {
                case 1:
                    System.out.print("Enter element to insert: ");
                    int insertKey = sc.nextInt();
                    hashTable.insert(insertKey);
                    break;
                case 2:
                    System.out.print("Enter element to search: ");
                    int searchKey = sc.nextInt();
                    hashTable.search(searchKey);
                    break;
                case 3:
                    System.out.print("Enter element to delete: ");
                    int deleteKey = sc.nextInt();
                    hashTable.delete(deleteKey);
                    break;
                case 4:
                    hashTable.display();
                    break;
                case 5:
                    System.out.println("Exiting...");
                    sc.close();
                    return;
                default:
                    System.out.println("Invalid choice. Please try again.");
            }
        }
    }
}
```

OUTPUT:

INFERENCE:

VIVA QUESTIONS:

1. What is hashing?

2. What is a hash function?

3. What is collision in hashing?

4. What is linear probing?

5. Name any two collision handling techniques.

RESULT:

Ex.No:	10	Implement sorting algorithms (bubble sort, merge sort)
Date:		

AIM:

To implement sorting algorithms such as Bubble Sort and Merge Sort.

ALGORITHM

Step 1: Start the program.

Step 2: Read array elements.

Step 3: Apply Bubble Sort to sort the array. **Step 4:** Apply Merge Sort to sort the array. **Step 5:** Compare the sorted results.

Step 6: Display the sorted array.

Step 7: Stop the program.

PROGRAM

```
import java.util.*;
import java.util.Scanner;

public class SortingAlgorithms {

    // Bubble Sort implementation
    public static void bubbleSort(int[] arr) {
        int n = arr.length;
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - 1 - i; j++) {
                if (arr[j] > arr[j + 1]) {
                    // Swap arr[j] and arr[j+1]
```

```
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

// Merge Sort implementation
public static void mergeSort(int[] arr) {
    if (arr.length < 2) {
        return;
    }

    int mid = arr.length / 2;
    int[] left = new int[mid];
    int[] right = new int[arr.length - mid];

    // Copy data to left and right subarrays
    System.arraycopy(arr, 0, left, 0, mid);
    System.arraycopy(arr, mid, right, 0, arr.length - mid);

    // Recursively sort both halves
    mergeSort(left);
    mergeSort(right);

    // Merge sorted halves
    merge(arr, left, right);
}

// Merge function to combine two sorted arrays
public static void merge(int[] arr, int[] left, int[] right) {
    int i = 0, j = 0, k = 0;
```

```
    // Merge the arrays into the original array arr[]
    while (i < left.length && j < right.length) {
        if (left[i] <= right[j]) {
            arr[k++] = left[i++];
        } else {
            arr[k++] = right[j++];
        }
    }

    // Copy remaining elements from left[]
    while (i < left.length) {
        arr[k++] = left[i++];
    }

    // Copy remaining elements from right[]
    while (j < right.length) {
        arr[k++] = right[j++];
    }
}

// Function to display an array
public static void displayArray(int[] arr) {
    for (int num : arr) {
        System.out.print(num + " ");
    }
    System.out.println();
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    // Read array size
```

```
System.out.print("Enter the number of elements: ");
int n = sc.nextInt();

int[] arr1 = new int[n];
int[] arr2 = new int[n];

// Read array elements
System.out.println("Enter the array elements: ");
for (int i = 0; i < n; i++) {
    int element = sc.nextInt();
    arr1[i] = element;  // Array for Bubble Sort
    arr2[i] = element;  // Array for Merge Sort
}

// Bubble Sort
bubbleSort(arr1);
System.out.println("Array sorted using Bubble Sort: ");
displayArray(arr1);

// Merge Sort
mergeSort(arr2);
System.out.println("Array sorted using Merge Sort: ");
displayArray(arr2);

// Comparison of the results
System.out.println("Comparison: Are both arrays equal?");
boolean areEqual = true;
for (int i = 0; i < n; i++) {
    if (arr1[i] != arr2[i]) {
        areEqual = false;
        break;
    }
}
```

```
        if (areEqual) {
            System.out.println("Yes, both arrays are identical.");
        } else {
            System.out.println("No, the arrays are not identical.");
        }

        sc.close();
    }
}
```

OUTPUT:

INFERENCE:

VIVA QUESTIONS:

1. What is sorting?

2. What is Bubble Sort?

3. What is the time complexity of Bubble Sort?

4. What is Merge Sort?

5. Which sorting algorithm is faster for large datasets?

RESULT: